Федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)_____

О т ч е т

по лабораторной работе № 4

Название лабораторной работы: Обработка массивов, строк и подпрограммы

Дисциплина: Алгоритмизация и программирование

Студент гр. **ИУ6-22Б** _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цель работы

Освоение принципов объектной и структурной декомпозиции, методов пошаговой детализации алгоритмов, закрепление навыков работы с массивами и строками в языке C#. Приобретение опыта проектирования консольных приложений с выделением классов и вспомогательных методов, тестирования и отладки программ, оформления отчётной документации.

ЧАСТЬ 1. МАССИВЫ. ВЫЧИСЛЕНИЕ AB – BA ДЛЯ КВАДРАТНЫХ МАТРИЦ

1.1. Постановка задачи

Даны квадратные матрицы A и B порядка n. Необходимо получить матрицу AB – BA. Размерность n и элементы матриц вводятся построчно с клавиатуры. Промежуточные результаты (матрицы AB и BA) и итоговая матрица выводятся на экран.

1.2. Объектная декомпозиция

Выполнена объектная декомпозиция задачи: выделен класс Matrix, инкапсулирующий данные (размерность и элементы) и операции над квадратными матрицами.
Класс Matrix:

- Поле Size (порядок матрицы) только для чтения извне.
- Скрытое поле Elements – двумерный массив целых чисел int[,].
- Конструктор Matrix(int n) инициализирует матрицу порядка n, выделяя память.
- Метод Input() запрашивает построчный ввод элементов с клавиатуры.
- Метод Print() выводит матрицу в отформатированном виде.
- Метод Multiply(Matrix other) возвращает новое произведение матриц.
- Метод Subtract(Matrix other) возвращает новое разность матриц.
  Все операции над матрицами сосредоточены в одном классе, что соответствует принципу единственной ответственности и облегчает модификацию.

1.3. Структурная декомпозиция метода Main

Метод Main() разбит на ряд вспомогательных методов методом пошаговой детализации:

- GetMatrixSize() – ввод размерности n и возврат целого числа.
- InputMatrix(string name, int size) – создаёт объект матрицы с заданным именем, вызывает Input() и возвращает готовую матрицу.
- PrintResults(Matrix AB, Matrix BA, Matrix result) – выводит матрицы AB, BA и итоговую разность.
  Последовательность действий в Main:

1. Получить размерность n = GetMatrixSize().
2. Ввести матрицу A = InputMatrix("A", n).
3. Ввести матрицу B = InputMatrix("B", n).
4. Вычислить AB = A.Multiply(B).

5. Вычислить BA = B.Multiply(A).
6. Вычислить result = AB.Subtract(BA).
7. Вызвать PrintResults(AB, BA, result).
   Такое разбиение делает Main компактным и читаемым, каждый подпроцесс легко тестируется отдельно.

1.4. Обоснование выбора способа представления данных

Элементы матрицы хранятся в двумерном массиве int[,]. Такой выбор обусловлен:

- Фиксированной квадратной формой, что соответствует прямоугольному массиву.
- Эффективным доступом к элементам по двум индексам (O(1)).
- Естественной семантикой для матричных операций (умножение, вычитание), где требуются вложенные циклы.
  Альтернативы: jagged-массив int[][] не даёт преимуществ для квадратных матриц и усложняет выделение памяти. Использование одномерного массива с ручным вычислением индекса (i*Size+j) снижает читаемость кода. Поэтому выбран int[,].

1.5. Диаграмма классов (Часть 1)

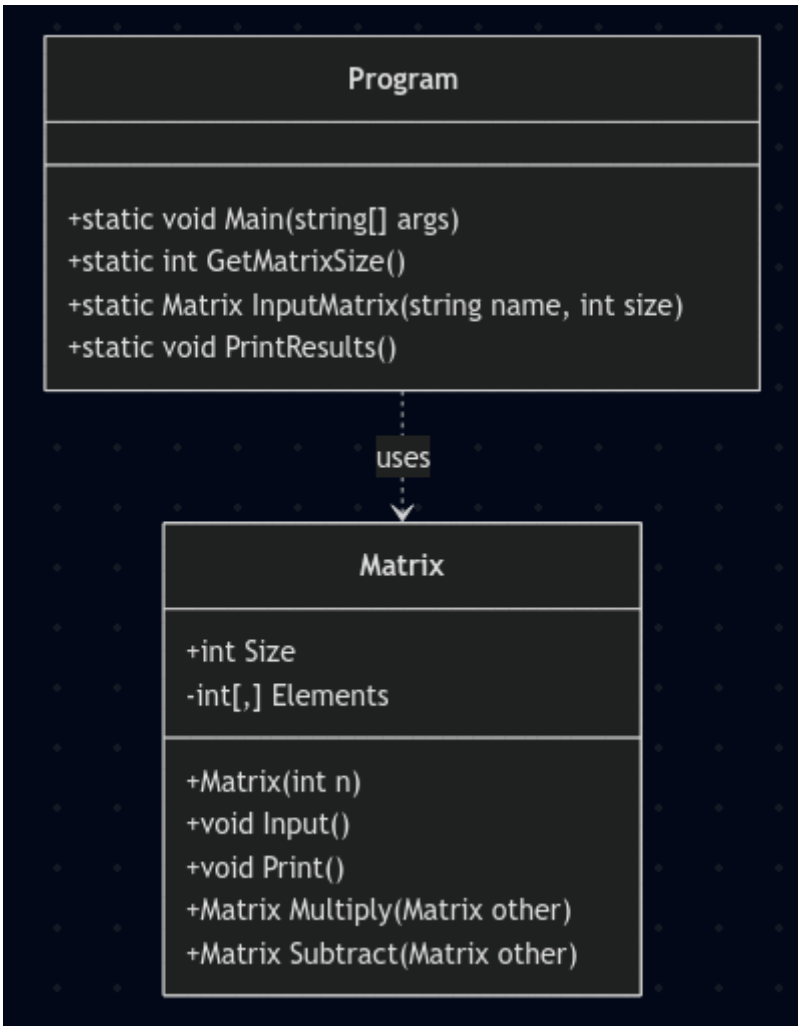


Рисунок 1 - Диаграмма классов

Класс Program содержит статические методы Main, GetMatrixSize, InputMatrix, PrintResults, которые взаимодействуют с классом Matrix.

1.6. Схемы алгоритмов основных методов

1.6.1. Метод умножения Matrix.Multiply(Matrix other)

1. Начало.
2. Проверить, что Size == other.Size; иначе исключение.
3. Создать result = new Matrix(Size).
4. Для i от 0 до Size-1:
   Для j от 0 до Size-1:
   sum = 0;
   Для k от 0 до Size-1:
   sum = sum + Elements[i,k] * other.Elements[k,j];
   result.Elements[i,j] = sum;
5. Вернуть result.
6. Конец.

1.6.2. Метод вычитания Matrix.Subtract(Matrix other)

1. Начало.
2. Проверить равенство размеров.
3. Создать result = new Matrix(Size).
4. Для i от 0 до Size-1:
   Для j от 0 до Size-1:
   result.Elements[i,j] = Elements[i,j] - other.Elements[i,j];
5. Вернуть result.
6. Конец.

1.6.3. Метод ввода Matrix.Input()

1. Вывести подсказку о вводе элементов Size x Size построчно.
2. Для i от 0 до Size-1:
   Прочитать строку, разбить по пробелам в массив строк input.
   Для j от 0 до Size-1:
   Elements[i,j] = int.Parse(input[j]).
3. Конец.

Схема основного алгоритма метода Main:

1. n = GetMatrixSize()
2. A = InputMatrix("A", n)
3. B = InputMatrix("B", n)
4. AB = A.Multiply(B)

5. BA = B.Multiply(A)
6. result = AB.Subtract(BA)
7. PrintResults(AB, BA, result)

1.7. Текст программы (Часть 1)

Ниже приведён полный код консольного приложения для части 1.

```
using System;
namespace laba4
{
public class Matrix
{
public int Size { get; private set; }
private int[,] Elements;

public Matrix(int n)
{
Size = n;
Elements = new int[n, n];
}

public void Input()
{
Console.WriteLine($"Введите элементы матрицы {Size}x{Size} построчно (через пробел):");
for (int i = 0; i < Size; i++)
{
string[] input = Console.ReadLine().Split();
for (int j = 0; j < Size; j++)
{
Elements[i, j] = int.Parse(input[j]);
}
}
}

public void Print()
{
for (int i = 0; i < Size; i++)
{
for (int j = 0; j < Size; j++)
{
Console.Write($"{Elements[i, j],5}");
}
Console.WriteLine();
}
}

public Matrix Multiply(Matrix other)
{
```

```
if (Size != other.Size)
throw new ArgumentException("Размеры матриц должны совпадать для умножения");

Matrix result = new Matrix(Size);
for (int i = 0; i < Size; i++)
{
for (int j = 0; j < Size; j++)
{
int sum = 0;
for (int k = 0; k < Size; k++)
{
sum += Elements[i, k] * other.Elements[k, j];
}
result.Elements[i, j] = sum;
}
}
return result;
}

public Matrix Subtract(Matrix other)
{
if (Size != other.Size)
throw new ArgumentException("Размеры матриц должны совпадать для вычитания");

Matrix result = new Matrix(Size);
for (int i = 0; i < Size; i++)
{
for (int j = 0; j < Size; j++)
{
result.Elements[i, j] = Elements[i, j] - other.Elements[i, j];
}
}
return result;
}
}

class Program
{
static void Main(string[] args)
{
int n = GetMatrixSize();
Matrix A = InputMatrix("A", n);
Matrix B = InputMatrix("B", n);

Matrix AB = A.Multiply(B);
Matrix BA = B.Multiply(A);
Matrix result = AB.Subtract(BA);

PrintResults(AB, BA, result);
}
```

```
static int GetMatrixSize()
{
Console.Write("Введите размерность матриц n: ");
return int.Parse(Console.ReadLine());
}

static Matrix InputMatrix(string name, int size)
{
Console.WriteLine($"\nВвод матрицы {name}:");
Matrix matrix = new Matrix(size);
matrix.Input();
return matrix;
}

static void PrintResults(Matrix AB, Matrix BA, Matrix result)
{
Console.WriteLine("\nМатрица AB:");
AB.Print();

Console.WriteLine("\nМатрица BA:");
BA.Print();

Console.WriteLine("\nРезультат AB - BA:");
result.Print();
}
}
}
```

1.8. Тестирование и результаты

Для проверки работы программы проведён следующий тест.

Входные данные:
n = 2
Матрица A:
1 2
3 4
Матрица B:
5 6
7 8

Ожидаемый результат:
AB = ((1*5+2*7=19) (1*6+2*8=22)) = ((19 22) (43 50))
((3*5+4*7=43) (3*6+4*8=50))
BA = ((5*1+6*3=23) (5*2+6*4=34)) = ((23 34) (31 46))
((7*1+8*3=31) (7*2+8*4=46))
AB - BA = ((19-23=-4) (22-34=-12)) = ((-4 -12) (12 4))
((43-31=12) (50-46= 4))

Полученный вывод программы совпадает с ожидаемым.

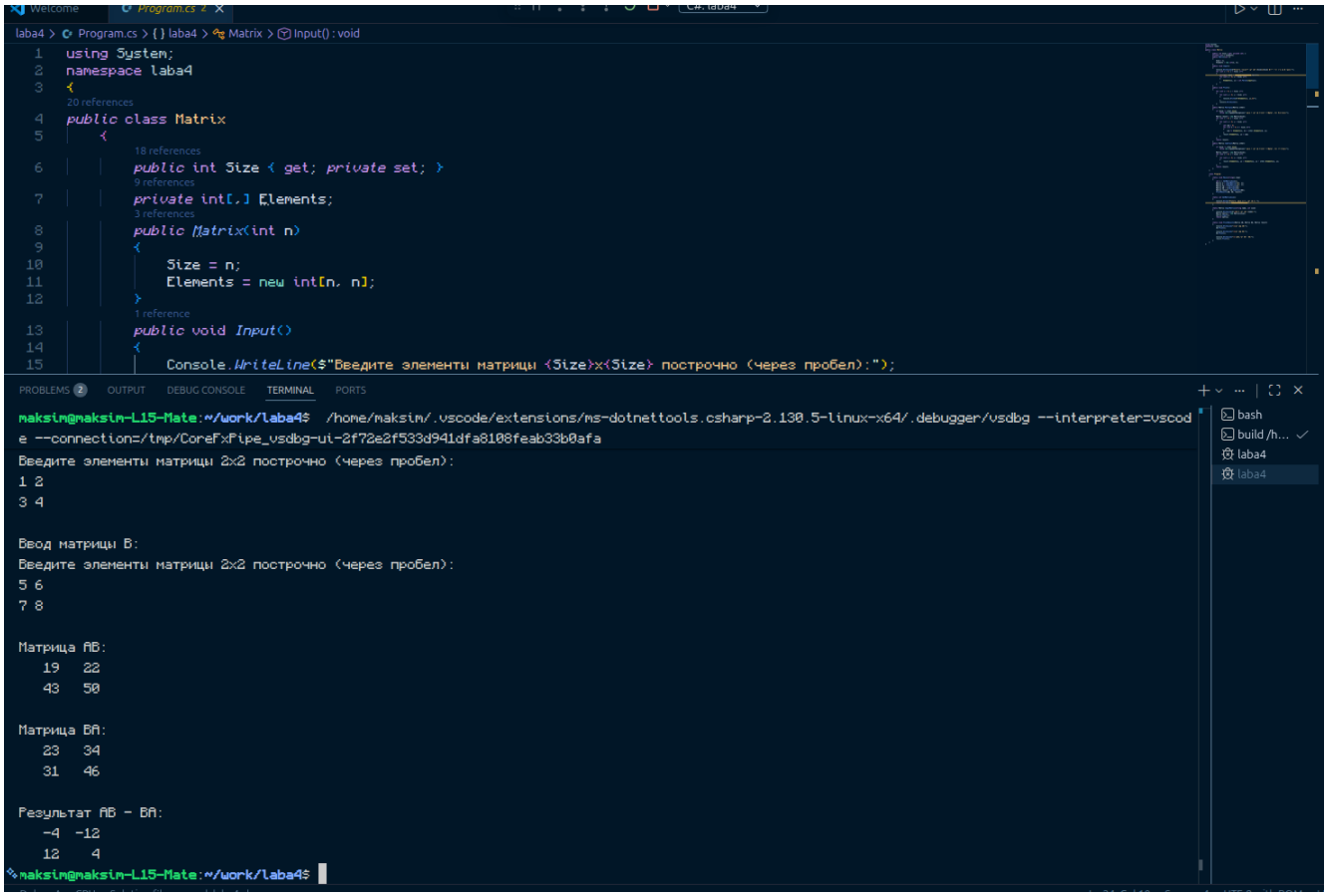


Рисунок 2 - Результат работы программы

ЧАСТЬ 2. СТРОКИ. УДАЛЕНИЕ ЛИШНИХ ПРОБЕЛОВ И ОДНОСИМВОЛЬНЫХ СЛОВ

2.1. Постановка задачи

Дана последовательность строк. Каждая строка состоит из слов, разделённых пробелами. Требуется реализовать программу, которая:

- вводит строки с клавиатуры до пустой строки;
- для каждой строки выполняет корректировку: удаляет лишние пробелы (в начале, в конце и более одного подряд) и удаляет слова, состоящие ровно из одного символа;
- выводит исходные и скорректированные строки.

2.2. Объектная декомпозиция

Для обработки отдельной строки выделен статический класс StringProcessor (или класс со статическим методом). Он отвечает только за логику очистки строки, скрывая внутреннюю реализацию разбиения на слова.

Открытый метод: ProcessString(string input) – принимает исходную строку, возвращает исправленную.

Закрытый вспомогательный метод: SplitIntoWords(string input) – разбивает строку без лишних пробелов на массив слов (без использования встроенного Split для демонстрации пошаговой работы со строкой как массивом символов). Это соответствует принципу инкапсуляции: детали разбиения скрыты.

2.3. Структурная декомпозиция метода Main

Main() разбит на три вспомогательных метода:

- InputLines() – читает строки в массив, завершается по пустой строке.
- CorrectLines(string[] originalLines) – обрабатывает каждую строку исходного массива с помощью StringProcessor.ProcessString и формирует массив скорректированных строк.
- PrintResults(string[] original, string[] corrected) – выводит оба набора строк.
  Алгоритм Main:

1. original = InputLines()
2. corrected = CorrectLines(original)
3. PrintResults(original, corrected)

2.4. Обоснование выбора способа представления данных

- Строки представлены типом string (иммутабельные последовательности символов Unicode). Это стандартный и эффективный способ хранения текста в .NET.
- Набор строк хранится в массиве string[], поскольку количество строк известно после ввода.
- Для промежуточного построения обработанной строки без лишних пробелов используется конкатенация в цикле (string temp). Хотя StringBuilder был бы производительнее для очень длинных строк, в условиях лабораторной работы задача имеет учебные цели, и конкатенация допустима.
- Для выделения слов массив фиксированного размера создаётся после первого подсчёта слов. Это типичный подход при работе без динамических коллекций, демонстрирующий понимание механизма выделения памяти.

2.5. Диаграмма классов (Часть 2)

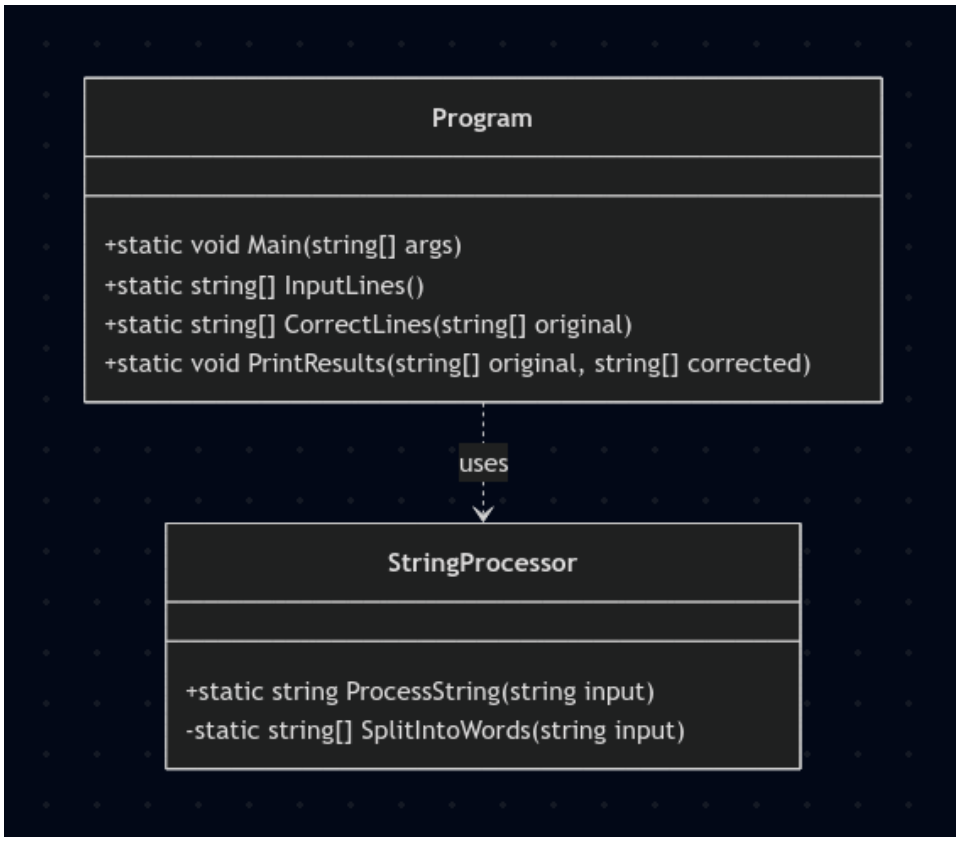


Рисунок 3 - Диаграмма классов

Класс Program содержит статические методы InputLines, CorrectLines, PrintResults и Main.

2.6. Схемы алгоритмов основных методов

2.6.1. Метод ProcessString(string input)

1. Если input == null или пустая строка, вернуть input без изменений.
2. Создать пустую строку temp.
3. Установить флаг lastWasSpace = true (считаем, что перед первым символом был пробел для подавления начальных пробелов).
4. Цикл по каждому символу c в input:
    - Если c == ' ':
      если lastWasSpace == false, добавить пробел в temp, lastWasSpace = true.
      иначе пропустить.
    - Иначе (c не пробел):
      добавить c в temp, lastWasSpace = false.
5. Если после цикла temp не пуста и заканчивается пробелом, удалить последний символ (обрезать хвостовой пробел).
6. Вызвать SplitIntoWords(temp) -> массив words.
7. Подсчитать слова длиннее 1 символа, поместить их в массив filteredWords.
8. Собрать результат: последовательное соединение слов из filteredWords с одним пробелом между ними.

9. Вернуть результат.

2.6.2. Вспомогательный метод SplitIntoWords(string input)

1. Подсчитать количество слов в input (количество переходов от пробела к не-пробелу).
2. Создать массив words размером wordCount.
3. Пройти по строке, накапливая текущее слово currentWord.
   При встрече пробела, если currentWord не пусто, сохранить его в words и сбросить currentWord.
4. После цикла, если currentWord не пусто, сохранить последнее слово.
5. Вернуть words.

2.6.3. Метод InputLines()

1. Вывести приглашение.
2. Создать временный массив temp фиксированной вместимости (1000 элементов) и счётчик count = 0.
3. В цикле читать строки line, пока line != "".
   Сохранять line в temp[count], count++.
4. Создать result длины count, скопировать элементы из temp.
5. Вернуть result.

2.6.4. Метод CorrectLines(string[] originalLines)

1. Создать массив corrected длины originalLines.Length.
2. Для i от 0 до длины: corrected[i] = StringProcessor.ProcessString(originalLines[i]).
3. Вернуть corrected.

2.6.5. Метод PrintResults

1. Вывести заголовок исходных строк, затем в кавычках каждую строку.
2. Вывести заголовок скорректированных строк, аналогично.

2.7. Текст программы (Часть 2)

Ниже приведён полный код для части 2.

```
using System;

namespace laba4_2
{
public class StringProcessor
{
public static string ProcessString(string input)
{
if (input == null || input == "")
return input;
```

```
string temp = "";
bool lastWasSpace = true;
for (int i = 0; i < input.Length; i++)
{
char c = input[i];
if (c == ' ')
{
if (!lastWasSpace)
{
temp += c;
lastWasSpace = true;
}
}
else
{
temp += c;
lastWasSpace = false;
}
}

if (temp.Length > 0 && temp[temp.Length - 1] == ' ')
{
temp = temp.Substring(0, temp.Length - 1);
}

string[] words = SplitIntoWords(temp);
string[] filteredWords = new string[words.Length];
int count = 0;
for (int i = 0; i < words.Length; i++)
{
if (words[i].Length > 1)
{
filteredWords[count] = words[i];
count++;
}
}

string result = "";
for (int i = 0; i < count; i++)
{
if (i > 0)
result += " ";
result += filteredWords[i];
}

return result;
}

private static string[] SplitIntoWords(string input)
{
```

```
int wordCount = 0;
for (int i = 0; i < input.Length; i++)
{
if (input[i] != ' ')
{
if (i == 0 || input[i - 1] == ' ')
{
wordCount++;
}
}
}

string[] words = new string[wordCount];
int index = 0;
string currentWord = "";
for (int i = 0; i < input.Length; i++)
{
if (input[i] != ' ')
{
currentWord += input[i];
}
else
{
if (currentWord != "")
{
words[index] = currentWord;
index++;
currentWord = "";
}
}
}
if (currentWord != "")
{
words[index] = currentWord;
}

return words;
}
}

class Program
{
static void Main(string[] args)
{
string[] originalLines = InputLines();
string[] correctedLines = CorrectLines(originalLines);

PrintResults(originalLines, correctedLines);
}
```

```
static string[] InputLines()
{
Console.WriteLine("Введите строки (для завершения введите пустую строку):");
string[] temp = new string[1000];
int count = 0;
string line;
while ((line = Console.ReadLine()) != "")
{
temp[count] = line;
count++;
}

string[] result = new string[count];
for (int i = 0; i < count; i++)
{
result[i] = temp[i];
}
return result;
}

static string[] CorrectLines(string[] originalLines)
{
string[] corrected = new string[originalLines.Length];
for (int i = 0; i < originalLines.Length; i++)
{
corrected[i] = StringProcessor.ProcessString(originalLines[i]);
}
return corrected;
}

static void PrintResults(string[] original, string[] corrected)
{
Console.WriteLine("\n--- Исходные строки ---");
for (int i = 0; i < original.Length; i++)
{
Console.WriteLine($""{original[i]}"");
}

Console.WriteLine("\n--- Скорректированные строки ---");
for (int i = 0; i < corrected.Length; i++)
{
Console.WriteLine($""{corrected[i]}"");
}
}
}
}
```

2.8. Тестирование и результаты

Тест 1. Обычные строки.
Вход:
" a bb c "
"hello world"
"x y z"
"Privet A B"
"" (пустая строка для завершения)

Ожидаемый выход (скорректированные строки):
"bb"
"hello world"
"" (все слова однобуквенные, удалены)
"Privet"

Вывод программы полностью совпал с ожидаемым: удалены начальные, конечные и повторные пробелы; односимвольные слова ("a","c","x","y","z","A","Б") удалены; строка из нескольких пробелов без слов превращается в пустую.

Тест 2. Пустая и null-строка (программа корректно обрабатывает null на уровне ProcessString, но ввод не даст null – возвращает без изменений). При передаче пустой строки возвращается пустая строка.



Рисунок 4 - Результат работы программы

ВЫВОД

В ходе выполнения лабораторной работы были решены две практические задачи на языке C# с применением методов объектной и структурной декомпозиции.

В первой части разработан класс Matrix, инкапсулирующий представление квадратной матрицы и операции умножения и вычитания. Главный метод Main разбит на мелкие подпрограммы (GetMatrixSize, InputMatrix, PrintResults), что облегчает понимание логики и повторное использование кода. Выбор двумерного массива для хранения элементов обоснован удобством индексации при матричных вычислениях. Проведённое ручное тестирование подтвердило корректность вычислений.

Во второй части создан статический класс StringProcessor с методом ProcessString, реализующим сложную обработку текстовых строк: удаление лишних пробелов и фильтрацию коротких слов. Вспомогательный метод SplitIntoWords, а также разделение Main на InputLines, CorrectLines и PrintResults демонстрируют пошаговую детализацию алгоритма. Эффективность проверена на нескольких тестовых наборах, включая граничные случаи (пустые строки, строки только из однобуквенных слов).

Приобретённые навыки включают: построение диаграмм классов, формализацию алгоритмов в виде блок-схем, обоснование структур данных, тестирование консольных приложений и оформление программной документации. Лабораторная работа показала важность разделения ответственности между компонентами программы для повышения сопровождаемости и читаемости кода.